Data Structure

CSE 0612-2131

Chapter 01: Introduction

Instructor
Saifur Rahman
Lecturer
Dept. of CSE, NBIU
Contact: saifur.naim30@gmail.com

Abstract Data Type

ADT

- An Abstract Data Type (ADT) describes how data is viewed and the operations allowed, without focusing on implementation details.
- It defines a type and a set of operations for that type.
- ADTs hide internal implementation details from the user.

Example

An ADT for a list of integers might include operations like:

1. Insert a new integer at a specific position.
2. Check if the list is empty (returns true/false).
3. Get the total number of integers.
4. Find the maximum integer.
5. Update the list.
6. Sort the list.

All are logical concepts but to implement them physically, we need data structure.

Data Structures

A data structure is a scheme for organizing data in the memory of a computer in order to use it efficiently.

Some of the more commonly used data structures include lists, arrays, stacks, queues, heaps, trees, and graphs.

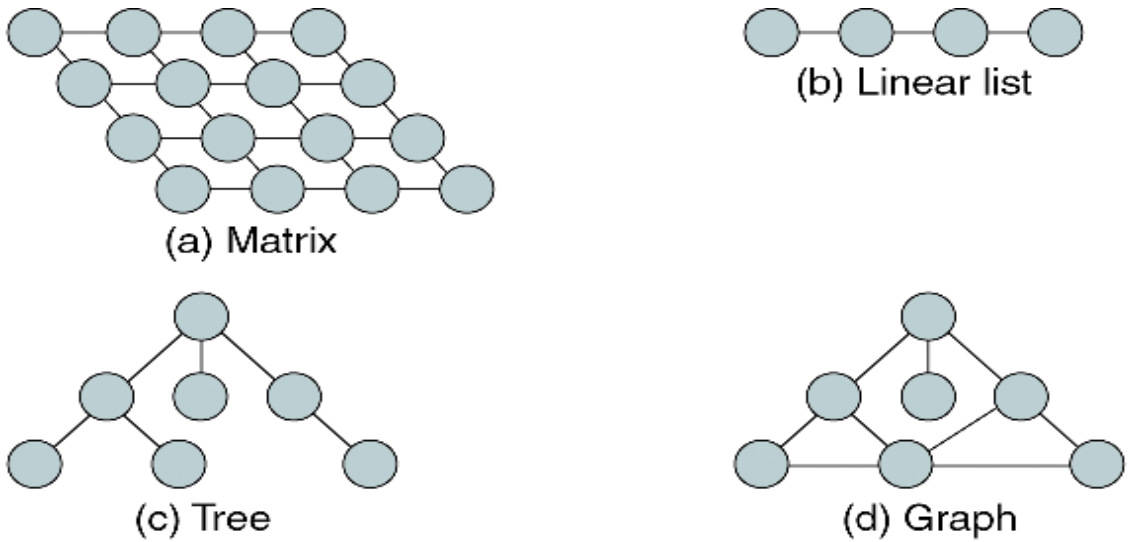


FIGURE 1-1 Some Data Structures

Data Structures: Preliminaries

- **Type**: A type is a collection of values.
  Example: Boolean, integer, character, floats etc.

- **Data Item**: A data item is a piece of information whose value is drawn from a type.

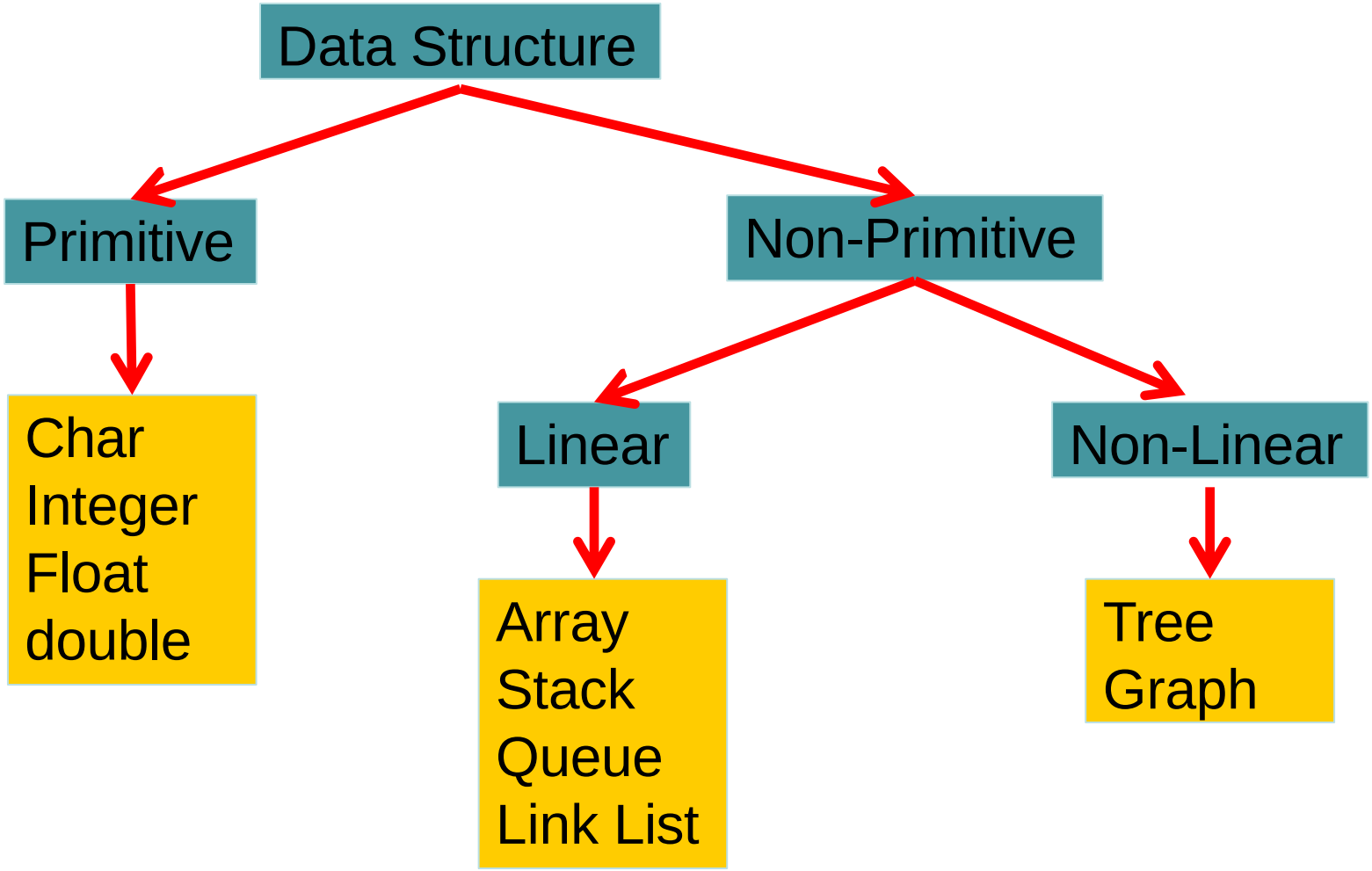
- **Record**: Set of data item.

- **File**: Set of Records.

Data Structure is the way in which data is organized, and it affects a program's performance for different tasks.

Computer programmers decide which data structures to use based on the nature of the data and the processes that need to be performed on that data.

Types of Data Structure



Linear and non Linear Data Structure

Linear data structure

A linear data structure traverses the data elements sequentially, allowing direct access to only one element.

Ex: Arrays, Linked Lists

Non-Linear data structure

Every data item is attached to several other data items in a way specific to reflecting relationships. The data items are not arranged in a sequential structure.

Ex: Trees, Graphs

Algorithms + Data Structures = Programs

Algorithms $\leftrightarrow$ Data Structures

Example: A Queue

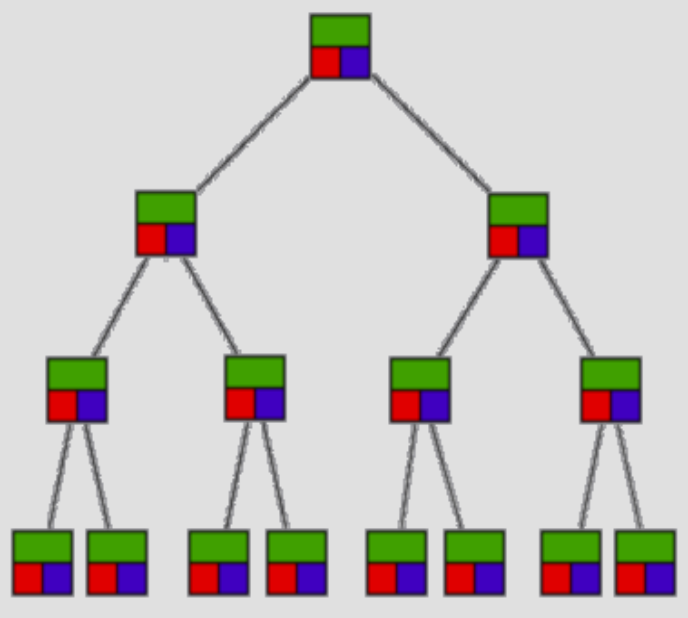
A *queue* is an example of commonly used simple data structure. A queue has beginning and end, called the *front* and *back* of the queue.

Data enters the queue at one end and leaves at the other. Because of this, data exits the queue in the same order in which it enters the queue, like people in a checkout line at a supermarket.

Example: A Binary Tree

A *binary tree* is another commonly used data structure. It is organized like an upside down tree.

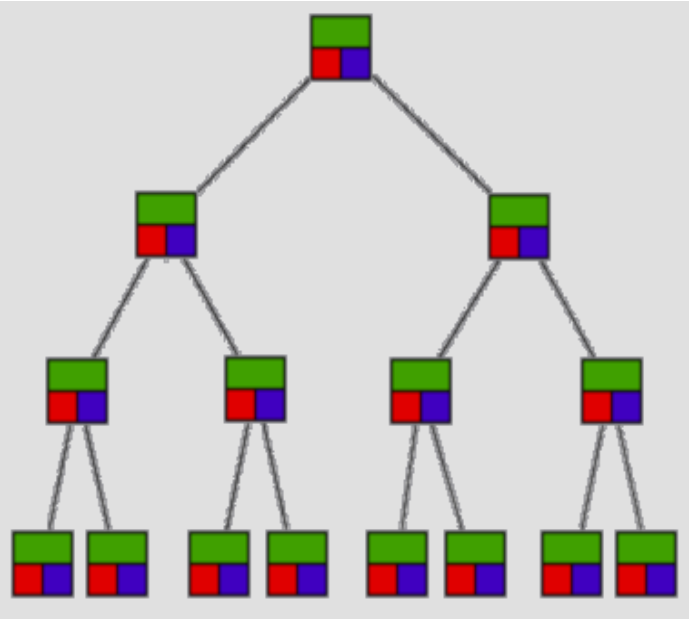
Each spot on the tree, called a *node*, holds an item of data along with a left pointer and a right pointer.



Binary Tree

Example: A Binary Tree (cont'd)

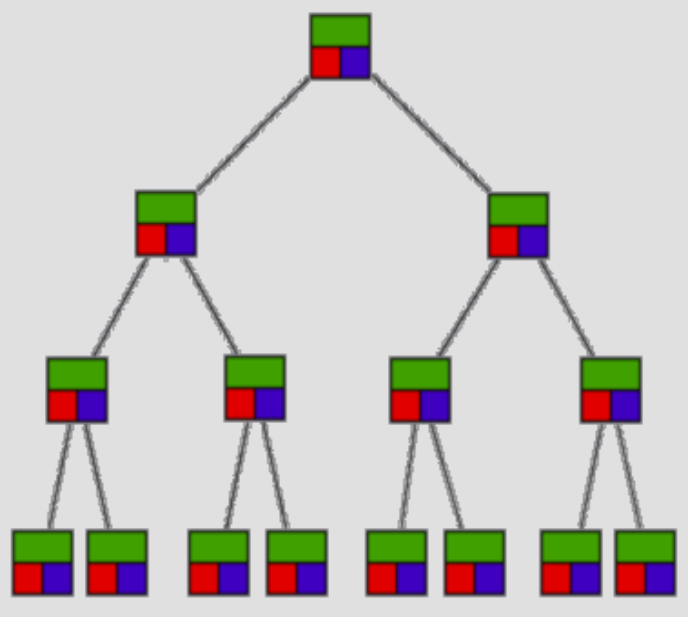
The pointers are lined up so that the structure forms the upside-down tree, with a single node at the top, called the root node, and branches increasing on the left and right as you go down the tree.



Binary Tree

Choosing Data Structures

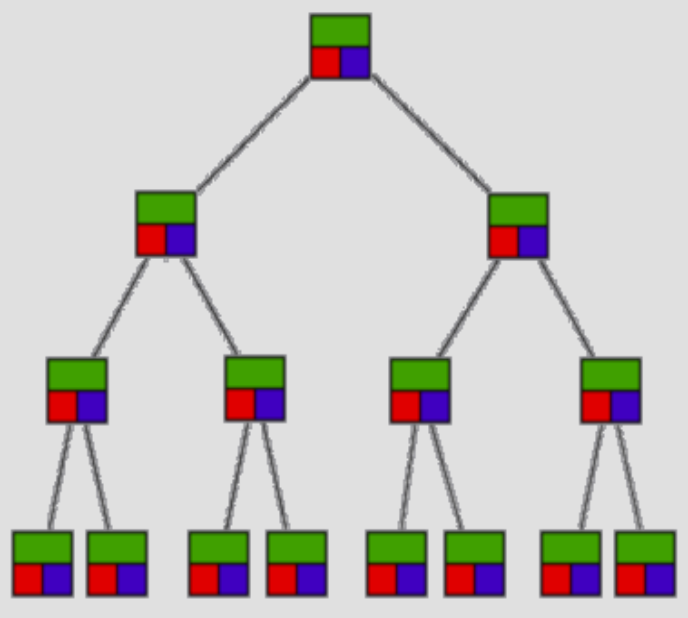
By comparing the queue with the binary tree, you can see how the structure of the data affects what can be done efficiently with the data.



Binary Tree

Choosing Data Structures (cont'd)

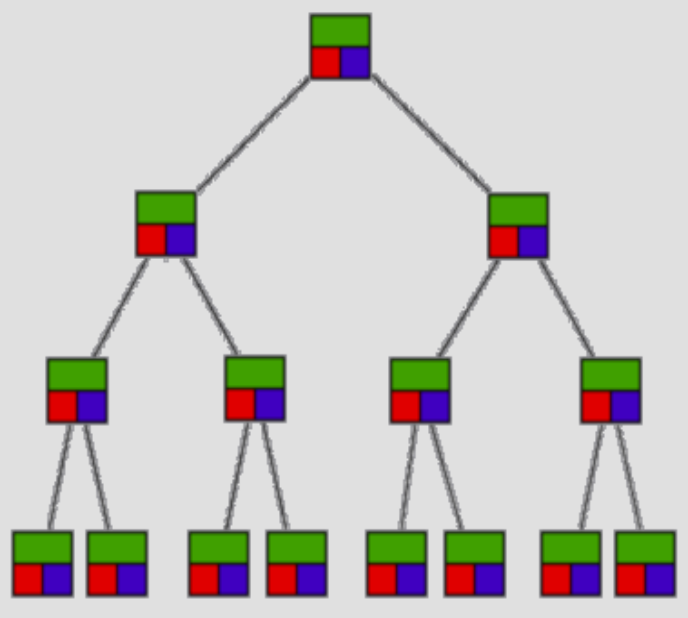
A queue is a good data structure to use for storing things that need to be kept in order, such as a set of documents waiting to be printed on a network printer.



Binary Tree

Choosing Data Structures (cont'd)

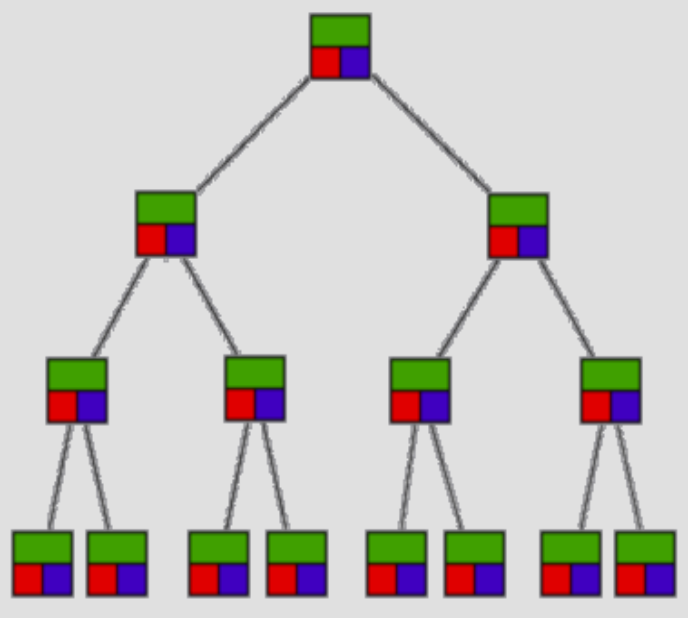
- The jobs will be printed in the order in which they are received.
- Most network print servers maintain such a *print queue*.



Binary Tree

Choosing Data Structures (cont'd)

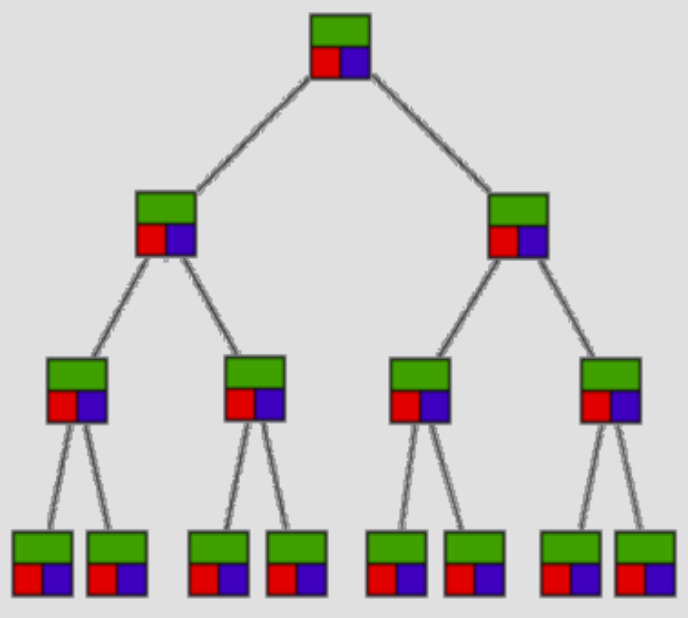
- A binary tree is a good data structure to use for searching sorted data.
- The middle item from the list is stored in the root node, with fewer items to the left and greater items to the right.



Binary Tree

Choosing Data Structures (cont'd)

- For some applications, a queue is the best data structure to use.
- For others, a binary tree is better.
- Programmers choose from among many data structures based on how the data will be used by the program.



Binary Tree

List & Arrays

List and arrays are two built-in data structures that can be used to organize data, or to create other data structures:

- Lists
- Arrays

List

Array

List & Arrays (cont'd)

- A list is an ordered set of data. It is often used to store objects that are to be processed sequentially.
- A list can be used to create a queue.

List

Array

List & Arrays (cont'd)

- An array is an indexed set of variables, such as $dancer_{[1]}$, $dancer_{[2]}$, $dancer_{[3]}$,… It is like a set of boxes that hold things.

- A list is a set of items.

- An array is a set of variables that each store an item.

List

Array

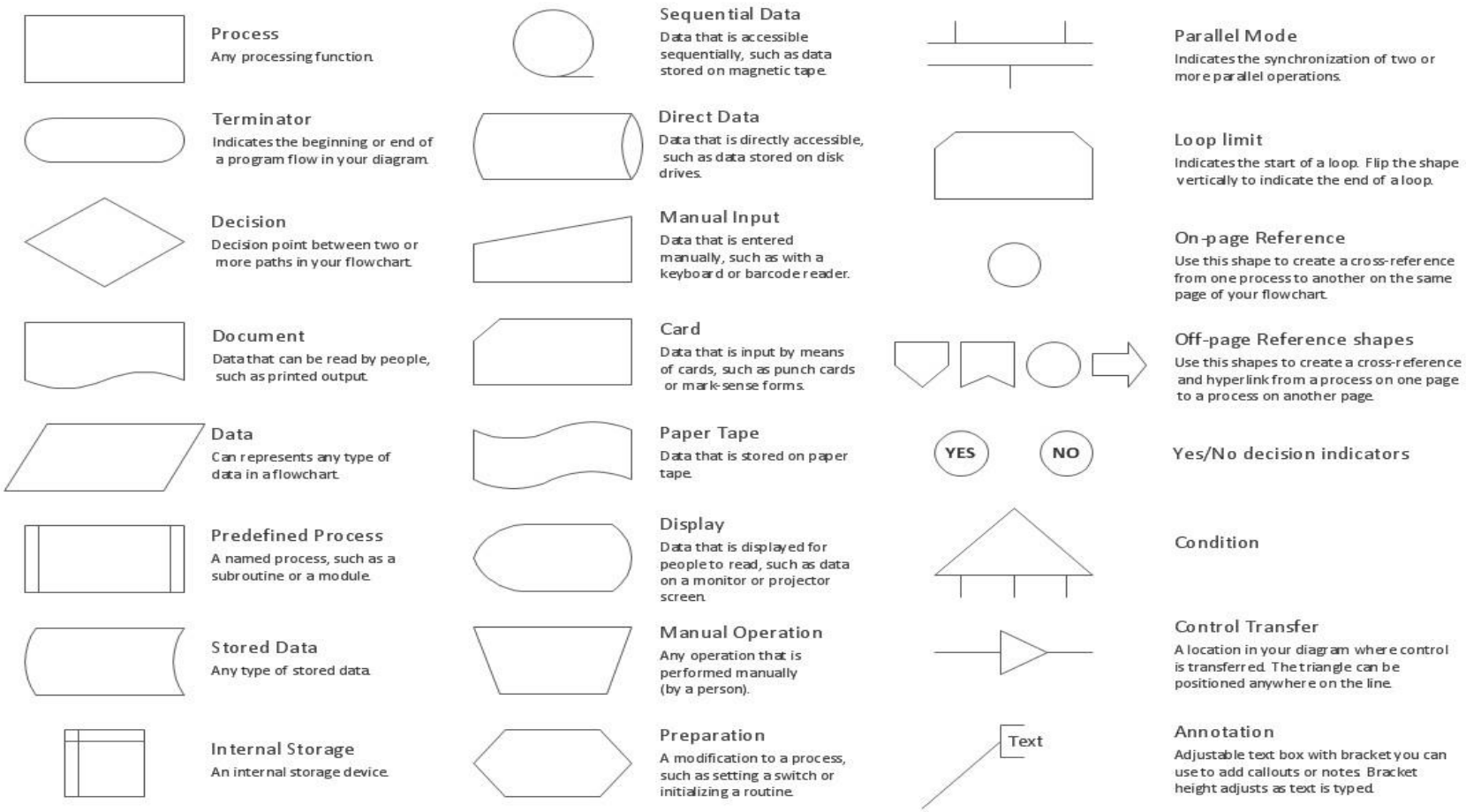
List & Arrays (cont'd)

You can see the difference between arrays and lists when you delete items.

Flow Chart

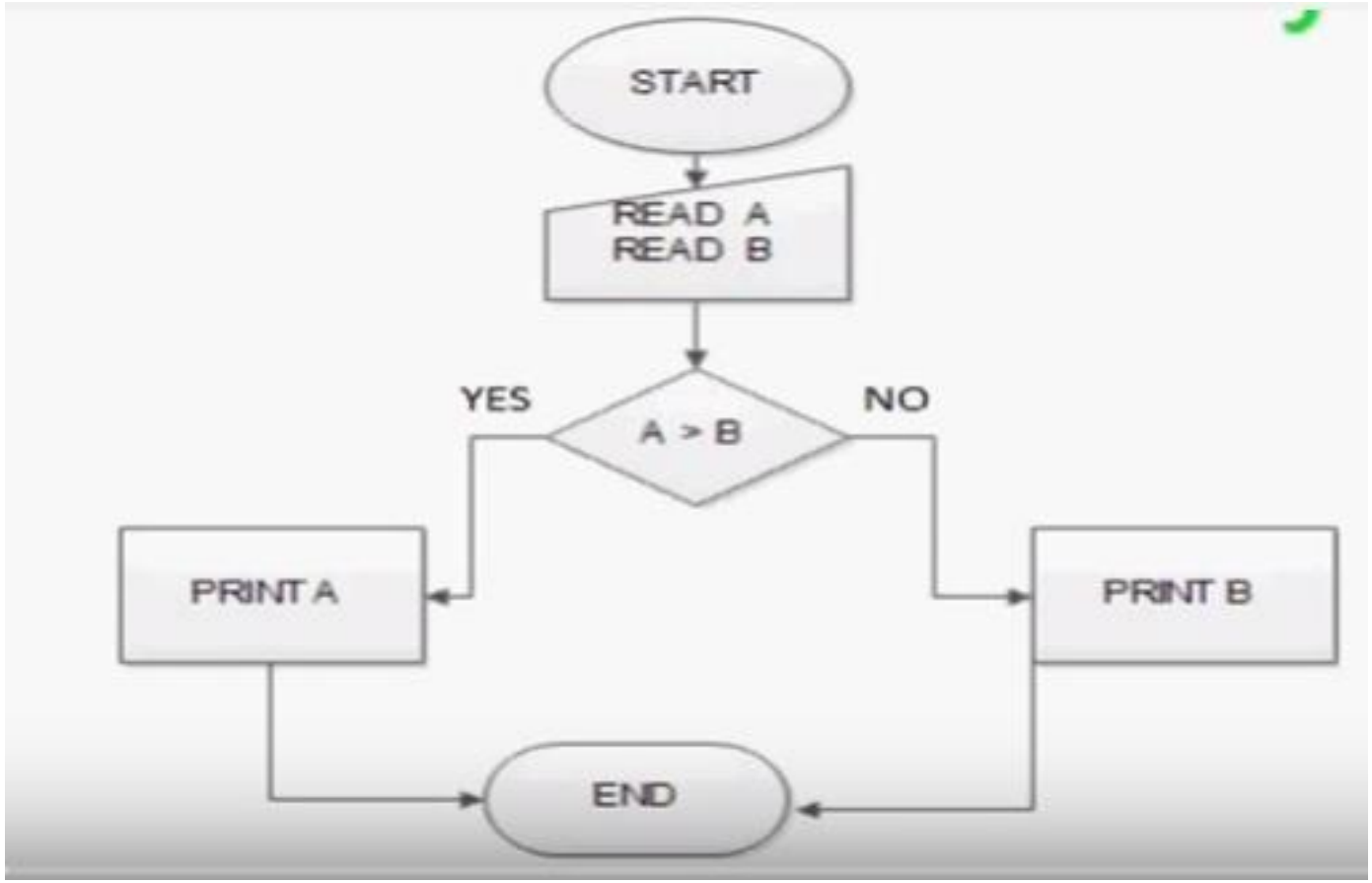
Flowchart is a pictorial presentation of an algorithm

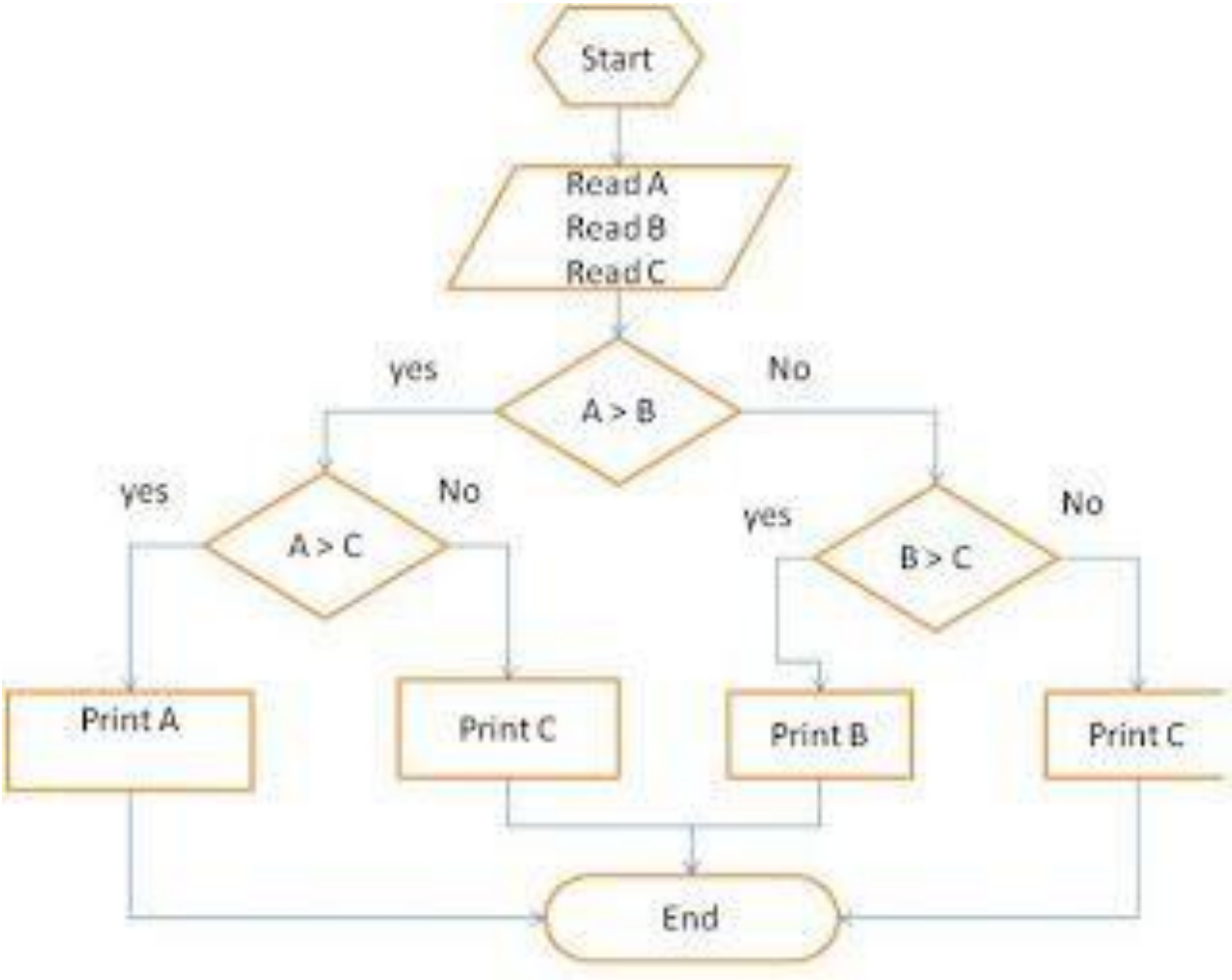
Design Elements - Cross-Functional Flowcharts solution - Flowcharts Shapes



Example

- Problem: Find the bigger between two numbers.



Example (cont'd)

- Problem: Find the bigger between three numbers.